Wasserstein Generative Adversarial Networks (WGANs)

Han Xiao

1 Introduction

Generative Adversarial Networks (GANs) are unsupervised deep learning models that use pre existing data to generate new, realistic data. The generated data has a variety of applications such as acting as training data for networks that lack sufficient data, performing image to image translations, 3D object generation, text to image translations, and more. In this paper, we will continue to talk about the specifics and training algorithm of Wasserstein Generative Adversarial Networks (WGANs), an improved model from basic GANs. For more information about the potential training issues, math, and other intricacies behind GANs, refer to the previous publication.

2 Wasserstein GAN Overview

The foundational structure of a generative adversarial network consists of a generator and a discriminator playing a minmax game with each other. The generator uses noise to produce artificial images that are fed into the discriminator along with images from a real dataset, while the discriminator attempts to assign the correct labels (eg. real or generated) to the received images. The minmax loss function can be simplified as

$$E_{z\sim p(x)}[\log D(x)] + E_{z\sim p(z)}[\log(1 - D(G(z)))]$$

Mathematically, the generator is trying to maximize $D(G(z))$, which ranges between zero and one, thus minimizing the overall function, while the discriminator attempts to do the opposite. Wasserstein networks differ in that they use a Wasserstein loss, or earth mover's distance to minimize function cost.

Furthermore, the discriminator contrasts the generator in that it is a convolutional network, which takes information directly from data, as opposed to a transposed convolutional network, which reconstructs data. As the name suggests, the discriminator reverses the process of the generator.

3 Discriminator: Identifying Data Attributes

A convolutional network is composed of convolutional layers, pooling layers, and a final fully connected layer, as shown below. In image processing, for instance, the convolutional layer consists of an array of weights that act as a small filter placed over a two-dimensional image. It calculates the dot products between the image's pixels, and this is repeated until the entire image has been run through the filter, creating the initial convolutional layer. Additional layers will stem from the first which focuses on different parts of the image. This is useful in situations where the object in the image can be viewed as the sum of its parts. Altogether, the convolutional layers

form a convolutional block. This block is then downsampled by a pooling layer, reducing the dimensionality from five by five to three by three, for example, through a filter of aggregation functions. Aggregation functions take all the features from a region, develop, and apply a singular global feature to the input. The pooling layer can either perform average pooling by calculating the receptive field's average value, or max pooling by applying the maximum value in the receptive field globally. Although this causes some information loss, it creates a more efficient network by reducing input complexity. Finally, the fully connected layer uses features from previous layers and classifies its input using a softmax activation function, which gives a probability between zero and one. This process can be thought of as the discriminator breaking down both generated and real images, and finally assigning a value to the image to classify how likely it is to be a real image.

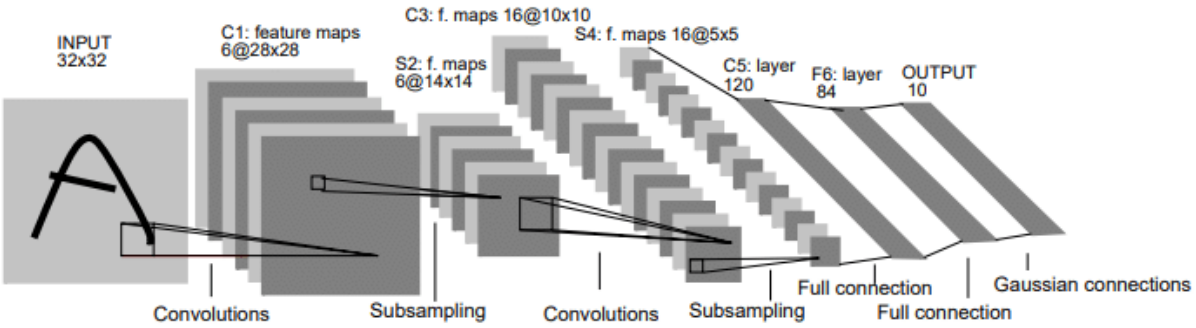
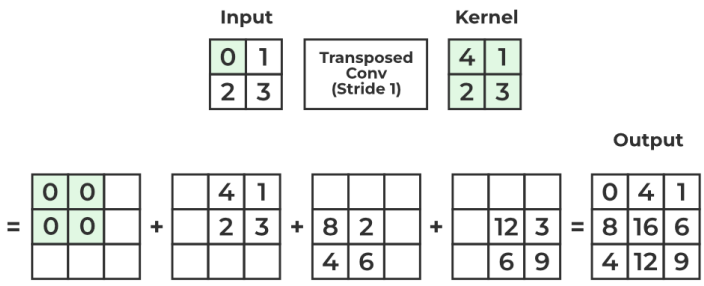


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

4 Generator

Transposed convolution networks, (re)construct images from their input data through upscaling. These networks are capable of producing high resolution images given a low resolution input and performing semantic segregation, which follows the idea previously mentioned: a whole is the sum of its parts. It will classify items in the image and visually label them with color.

After receiving an input with a certain dimension, a filter or kernel, usually a matrix with dimensions 2x2 or 3x3 will slide over the input and perform multiplication. The products of the values are then transferred over to a blank output matrix with the desired dimensions. To further explain using the diagram below, the values of the kernel are multiplied by the value of a pixel of the input and brought over to the output matrix (ex. 0x4, 0x1, 0x2, and 0x3). This is repeated with other pixels within the input before summing up the products that overlap.



It is important to note that transposed convolution layers are not the same as deconvolutional layers. Deconvolutional networks utilize the inverse of the multivariate convolutional function

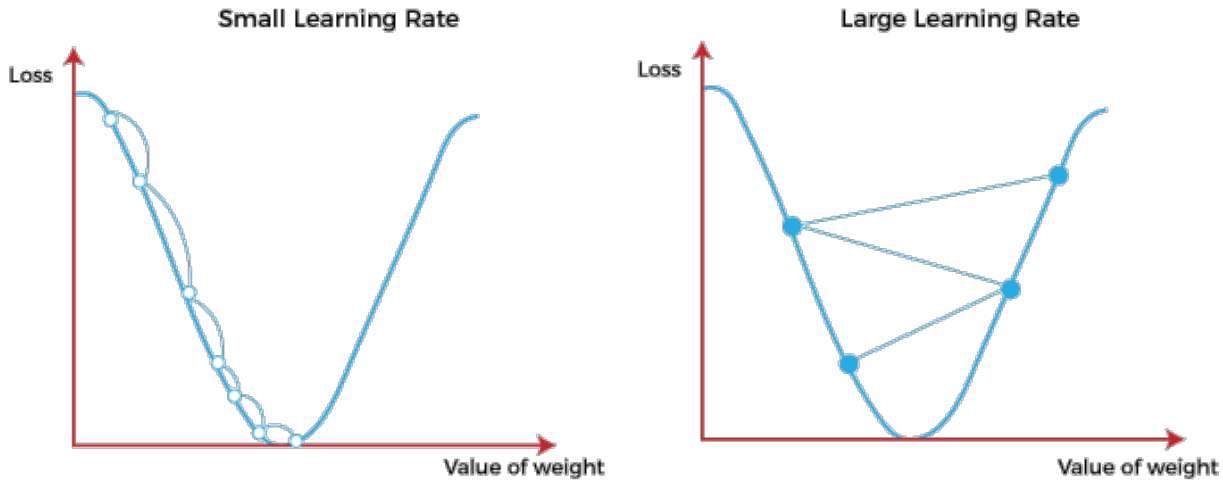
$$h = g \cdot f = \int_x \int_y g(t_1 - x, t_2 - y) f(x, y) dx dy$$

The output of a deconvolutional network is the same as its input. In other words, it will spit out the same image that it was given. On the other hand, transposed convolutional networks produce clearer images that are larger than the one that was input.

5 Network Training

In neural networks, there are hyperparameters such as learning rate, batch size, iterations, epochs, optimizers, and activation functions.

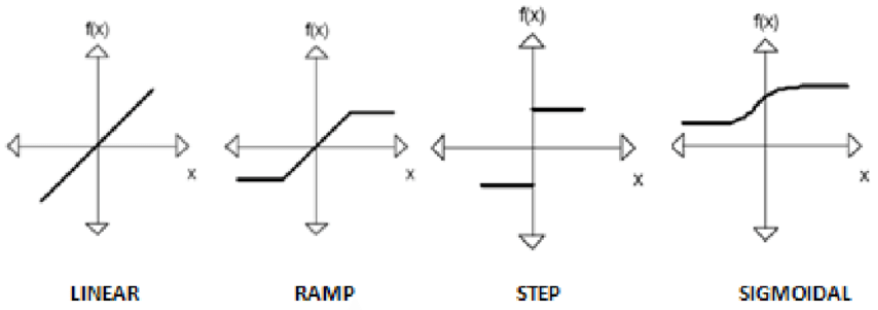
The learning rate is one of the most important hyperparameters for adjusting networks. It sets the pace at which weights are updated in order to decrease network loss. It is important to find an optimal learning rate, as rates that are too small can slow down training and fail to improve the loss function, and rates that are too large can cause divergence, prolonging the training process. To visualize the effects, we can imagine a point descending a curve to reach the global minimum, or convergence. If the learning rate is too small, the point descends too slowly. Contrastly, if the learning rate is too high, the point approaches the global minima quickly but has trouble reaching the minimum as it bounces around continuously. The process of reaching convergence, also known as gradient descent, can become more complicated if there are multiple local minima present in addition to the global minimum.



An epoch is the passing of an entire training set through the network. Usually, an epoch contains too much information to be inputted into the computer at once, so the data is divided into batches. Batch size indicates the number of training samples that should pass through the network before parameters like weights are updated, and iterations involve the number of batches that complete an epoch.

The activation function modifies inputted data to fit within a certain range of values, with the range depending on the type of function used, and establishes a threshold for neuron activation.

Common non-linear activation functions include the binary step function, which labels data as zeros and ones, the softmax function, ranging from zero to one and forcing the sum of all probabilities to equal to one, and the Rectified linear unit (ReLU) function with range zero to infinity.



In most models, a batch normalization layer is present before any activation function is applied. The layer zero-centers and normalizes the input using the batch's mean and standard deviation. Mathematically, the empirical mean of the batch is subtracted from the zero-centered and normalized input, and divided by the square root of the sum of the empirical standard deviation and some minuscule number (usually 10^{-5}, to avoid undefined numbers). Finally, this quotient is multiplied by the scaling parameter for the layer and added to a shifting or offset parameter. This layer further prevents vanishing/exploding gradient problems from arising during training.

6 WGAN Training Algorithm

To elaborate on where the last publication GANs left off, we will be breaking down the original Wasserstein Network algorithm and comparing it to an improved version. Overall, WGANs follow a similar network training process as mentioned above. However, there are a few tweaks to the algorithm other than the unique Wasserstein gradient penalty. The initial proposed algorithm is as follows:

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α, the learning rate. c, the clipping parameter. m, the batch size. n_{critic}, the number of iterations of the critic per generator iteration.
Require: : w_0, initial critic parameters. θ_0, initial generator's parameters.
while θ has not converged **do**
  for $t = 0, ..., n_{\text{critic}}$ **do**
    Sample $\{x^{(i)}\}_{i=1}^{m} \sim \mathbb{P}_r$ a batch from the real data.
    Sample $\{z^{(i)}\}_{i=1}^{m} \sim p(z)$ a batch of prior samples.
    $g_w \leftarrow \nabla_w \left[\frac{1}{m}\sum_{i=1}^{m} f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))\right]$
    $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$
    $w \leftarrow \text{clip}(w, -c, c)$
  end for
  Sample $\{z^{(i)}\}_{i=1}^{m} \sim p(z)$ a batch of prior samples.
  $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))$
  $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$
end while

The discriminator component uses simple linear activation functions in the form of $y = mx + b$, class labels of one and negative one are assigned for real and fake images, respectively, the Wasserstein loss is applied, weight clipping is applied, and parameters are tweaked with RMSProp gradient descent. It is also notable that the critic is updated more frequently than the generator, shown by the value of n_{critic}, which denotes five discriminator weight updates for every one update to the generator. Other lines of the algorithm show the mathematical processes that utilize the earth mover's distance.

Some components that distinguish this original algorithm from the rest are weight clipping, a method to keep the Lipschitz constant of the critic under control, and RMSProp (Root Mean Squared Propagation), an optimization algorithm. We see the use of weight clipping in line seven, which constrains all the parameters of the critic to certain set values. Although the proposers of this algorithm recognize the weaknesses of weight clipping, such as prolonging training, it was by far the simplest and best working method at the time of publication. RMSProp optimizes parameter values to make for a time efficient gradient descent and speed up convergence by using exponential decay to only accumulate gradients, or changes in weights, from the most recent iterations. RMSProp was shown to perform better than other optimizers at the time due to its ability to not slow down too fast, which may prevent convergence to the global optimum.

7 Improved WGAN Algorithm and Training

In a slightly newer publication, the authors show the negative effects of weight clipping and suggest using gradient penalty. In addition, Adam, a refined optimization algorithm, is used instead of RMSProp. This is visible in lines seven, nine, and twelve.

Algorithm 1 WGAN with gradient penalty. We use default values of $\lambda = 10$, $n_{\text{critic}} = 5$, $\alpha = 0.0001$, $\beta_1 = 0$, $\beta_2 = 0.9$.

Require: The gradient penalty coefficient λ, the number of critic iterations per generator iteration n_{critic}, the batch size m, Adam hyperparameters α, β_1, β_2.
Require: initial critic parameters w_0, initial generator parameters θ_0.
1: **while** θ has not converged **do**
2: **for** $t = 1, ..., n_{\text{critic}}$ **do**
3: **for** $i = 1, ..., m$ **do**
4: Sample real data $\boldsymbol{x} \sim \mathbb{P}_r$, latent variable $\boldsymbol{z} \sim p(\boldsymbol{z})$, a random number $\epsilon \sim U[0, 1]$.
5: $\tilde{\boldsymbol{x}} \leftarrow G_\theta(\boldsymbol{z})$
6: $\hat{\boldsymbol{x}} \leftarrow \epsilon \boldsymbol{x} + (1 - \epsilon)\tilde{\boldsymbol{x}}$
7: $L^{(i)} \leftarrow D_w(\tilde{\boldsymbol{x}}) - D_w(\boldsymbol{x}) + \lambda(\|\nabla_{\hat{\boldsymbol{x}}} D_w(\hat{\boldsymbol{x}})\|_2 - 1)^2$
8: **end for**
9: $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^{m} L^{(i)}, w, \alpha, \beta_1, \beta_2)$
10: **end for**
11: Sample a batch of latent variables $\{\boldsymbol{z}^{(i)}\}_{i=1}^{m} \sim p(\boldsymbol{z})$.
12: $\theta \leftarrow \text{Adam}(\nabla_\theta \frac{1}{m} \sum_{i=1}^{m} -D_w(G_\theta(\boldsymbol{z})), \theta, \alpha, \beta_1, \beta_2)$
13: **end while**

The authors of this publication showed that weight clipping, amongst other weight constraints, tend to lead to a failure to fully converge, and predisposes the discriminator towards simpler functions. The alternative way to enforce the Lipschitz constraint is to use gradient penalty, which focuses on constraints along coupled points from the real and generated images. The penalty contains a penalty coefficient that controls the rate of convergence through minimization. Within this change, batch normalization of the critic is removed, since gradient penalty focuses on mapping

individual inputs to individual outputs as opposed to a whole batch of inputs to a whole batch of outputs. Lastly, gradient descent encourages penalty of both the generator and the discriminator, or the norm of the gradient to approach a value of one. Although this has not been further investigated, it seems to improve the results of the network. Adam optimization is similar to RMSProp in that it keeps track of the exponential decay averages of past squared gradients and is an adaptive learning rate algorithm. However, it also tracks the exponential decay average of past gradients. Adam introduces more hyperparameters, β_1 and β_2.

Other changes include changing class label numbers from one and negative one to zero and one.

8 Conclusion

There are many other ways to improve Wasserstein Generative Adversarial networks, but the improvements may be more difficult to implement or have a high time and cost consumption. In the field of generative networks, it has been proven that if there is sufficient data, networks containing more basic components and computations are able to achieve the same results and even better ones than the improved algorithms we have now.

References

Arjovsky, Martin, Soumith Chintala, and Léon Bottou. "Wasserstein GAN," December 6, 2017. https://arxiv.org/pdf/1701.07875.pdf.

Boesch, Gaudenz. "Guide to Generative Adversarial Networks (GANs) in 2023." viso.ai, February 1, 2023. https://viso.ai/deep-learning/generative-adversarial-networks-gan/.

Brownlee, Jason. "How to Develop a Wasserstein Generative Adversarial Network (WGAN) from Scratch." Machine Learning Mastery, July 16, 2019. https://machinelearningmastery.com/how-to-code-a-wasserstein-generative-adversarial-network-wgan-from-scratch/.

Datagen. "Convolutional Neural Network: Benefits, Types, and Applications." Datagen, n.d. https://datagen.tech/guides/computer-vision/cnn-convolutional-neural-network/.

Doshi, Chinesh. "Generalized Way of Interpreting CNNs!!" Medium, August 22, 2020. https://medium.com/@chinesh4/generalized-way-of-interpreting-cnns-a7d1b0178709

Gulrajani, Ishaan, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. "Improved Training of Wasserstein GANs," December 25, 2017. https://arxiv.org/pdf/1704.00028.pdf.

Tsang, Sik-Ho. "Brief Review —WGAN-GP: Improved Training of Wasserstein GANs." Medium, August 1, 2023. https://sh-tsang.medium.com/brief-review-wgan-gp-improved-training-of-wasserstein-gans-ae3e2acb25b3.